FAESA
CENTRO UNIVERSITÁRIO

Disciplina: LINGUAGEM DE PROGRAMAÇÃO ORIENTADA A OBJETOS **Professor**: Cinthia Cristina Lucio Caliari

Nome do Aluno: .....

Nome do Aluno: .....

Nome do Aluno: .....

Nome do Aluno: .....

EXERCÍCIO AVALIATIVO DA C2

Observações:

i. Este código deve ser feito em grupo de até 4 pessoas
ii. Ele deverá ser postado no Envio de Trabalho até o dia 22/10 às 23:59.
iii. Valor: 4,0 pontos na C2
iv. O grupo deve desenvolver os códigos abaixo e postar no Envio de Trabalho.
v. O grupo deve postar também o relatório do secretário contendo a contribuição de cada componente do grupo no desenvolvimento dos exercícios

(4,0) 1 – Desenvolva um programa em Java que simule uma invasão de criaturas a um abrigo de sobreviventes.

Uma cidade foi invadida por criaturas perigosas, e um grupo de sobreviventes tenta proteger seu abrigo. Existem diferentes tipos de criaturas, cada uma com características e formas de ataque próprias.

O abrigo possui pontos de vida e defesas, que diminuem à medida que sofre os ataques.

Para implemente esse jogo, você vai precisa fazer as seguintes classes:

a) Abrigo

- Deverá possuir os atributos de instância:
  - **defesa:** resistência do abrigo;
  - **pontosVida**: de ataques que ainda podem ser bloqueados, que deve valer, no mínimo, zero;
  - **maxDefesa**: que dever ser um valor interno que armazena a quantidade de defesa recebida na sua instanciação.
- Métodos concretos:
  - **get** dos atributos defesa e pontosVida (não tem set de nenhum)
  - **estaDestruido**: que retorna true se o abrigo foi destruído ou false caso contrário. O abrigo será destruído quanto sua defesa seus pontos de vida forem iguais a zero.
  - **receberAtaque**: que deve receber o valor do dano e retirar dos pontos de vida. Lembrando que os pontos de vida devem valer, em qualquer momento do jogo, no mínimo, 0 (ZERO). Deve retornar true, se o abrigo for derrotada, isto é, se seus pontos de vida e sua defesa atingirem ZERO; ou false, caso contrário. Esse método deve atualizar os atributos necessários. Por exemplo, se o abrigo foi instanciado com defesa = 2 e pontos de vida = 20:
    - Se dano = 10, pontos de vida = 20 e defesa = 2, depois do ataque ficará: pontos de vida = 10 (20 - 10) e defesa = 2

- Se dano = 10, pontos de vida = 10 e defesa = 2, ficará: pontos de vida = 20 (10-10 = 0 e pontos de vida recupera os 20 pontos iniciais) e defesa = 1
- Se dano = 10, pontos de vida = 3 e defesa = 2, ficará: pontos de vida = 13 (3 – 10 = – 7. Como não pode ficar negativo, soma com o valor inicial de pontos de vida: -7 + 20 = 13) e defesa = 1
- Se dano = 10, pontos de vida = 5 e defesa = 0, ficará: pontos de vida = 0 e defesa = 0

b) Criatura (classe abstrata)

- Deverá possuir os atributos de instância:
  - **nome**: identificação da criatura;
  - **pontosVida**: quantidade de vida da criatura, que deve valer, no mínimo, zero;
  - **poderAtaque**: intensidade de seus ataques.
  - **derrotada**: que armazena true ou false, indicando se a criatura foi derrotada ou não.
- Atributo de classe:
  - **quantDerrotadas**: contabiliza a quantidade total de criaturas derrotadas durante a execução do jogo.
- Métodos concretos:
  - **get** de todos os atributos (não tem set de nenhum)
  - **receberDano**: que deve receber o valor do dano e retirar dos pontos de vida. Lembrando que os pontos de vida deve valer, em qualquer momento do jogo, no mínimo, 0 (ZERO). Deve retornar true, se a criatura for derrotada, isto é, se seus pontos de vida atingirem ZERO; ou false, caso contrário. Esse método deve atualizar os atributos necessários.
- Método abstrato:
  - **atacar(Abrigo abrigo):** que será implementado pelas subclasses.

c) Zumbi que herda de Criatura. Quando atacar, deverá causar ao abrigo um dano correspondente ao seu poder de ataque.

d) Mutante que herda de Criatura e tem maior capacidade de destruição, pois seu ataque deverá causar dano equivalente ao dobro de seu poder de ataque.

Regras do jogo

O grupo deverá implementar um aplicativo que:

1. Crie um abrigo com 100 pontos de vida e 5 defesas.

2. Crie 3 Zumbis, cada um com 30 pontos de vida e poder de ataque 10.

3. Crie 2 Mutantes, cada um com 20 pontos de vida e poder de ataque 15.

4. Armazene todas as criaturas em uma ArrayList<Criatura>.

5. A cada rodada, os sobreviventes vão atacar as criaturas. Assim, deverá ser sorteado um valor de dano entre 5 e 20, que será aplicado a uma criatura aleatória. Se os pontos de vida dessa criatura chegar a zero, ela será considerada derrotada e deverá ser removida da lista, atualizando o contador estático de criaturas derrotadas. Deverá ser mostrado na tela a seguinte mensagem:

Os sobreviventes atacaram!
Alvo: <colocar a classe do alvo (Zumbi ou Mutante) e o seu nome>
Dano causado: <colocar o valor do dano

--- SITUACAO DA CRIATURA ---
Nome: <colocar o nome>
Pontos de vida: <seus pontos de vida>
Poder de ataque: <seu poder de ataque>
Total de criaturas derrotadas: <seu valor>

6. Logo em seguida, uma criatura ainda ativa deverá ser escolhida aleatoriamente para atacar o abrigo. E deve escrever na tela uma mensagem semelhante à mensagem acima:
<nome de sua classe>: <seu nome> atacou o abrigo!
Dano do ataque: <valor>
O abrigo bloqueou o ataque! ou **O abrigo foi destruído**

--- SITUACAO DO ABRIGO ---
Pontos de vida: <valor>
Defesas restantes: <valor>

O jogo continuará enquanto houver criaturas na lista e o abrigo não tiver sido destruído.

Quando todas as criaturas forem derrotadas, o programa deverá exibir a mensagem **“ABRIGO DEFENDIDO — OS SOBREVIVENTES VENCERAM!”**.

Caso o abrigo seja derrotado, o programa deverá exibir a mensagem **“ABRIGO DESTRUÍDO — AS CRIATURAS VENCERAM!”**.

Ao final, deverá mostrar a quantidade total de criaturas derrotadas.

Orientações complementares para os alunos

- Utilize polimorfismo corretamente
-
- Organize o código em classes separadas e implemente construtores e métodos de acesso quando necessário.
-
- O atributo estático deve ser incrementado somente quando uma criatura for efetivamente derrotada, e cada criatura deve ser contabilizada uma única vez. O método de ataque deve respeitar as regras de cada subclasse.
- O programa deve rodar sem nenhum interferência do usuário, ou seja, não será perguntado nada ao usuário.